Unit- 2

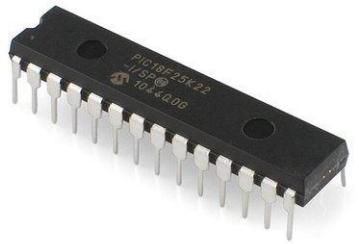
Sensors, Microcontrollers and their Interfacing

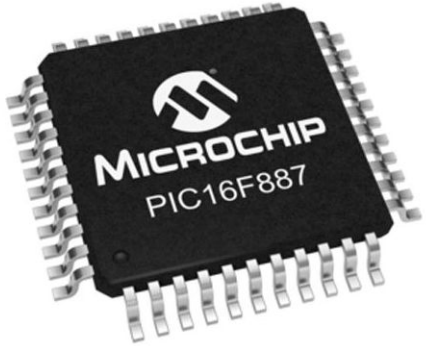
Microcontrollers

Overview of Microcontrollers

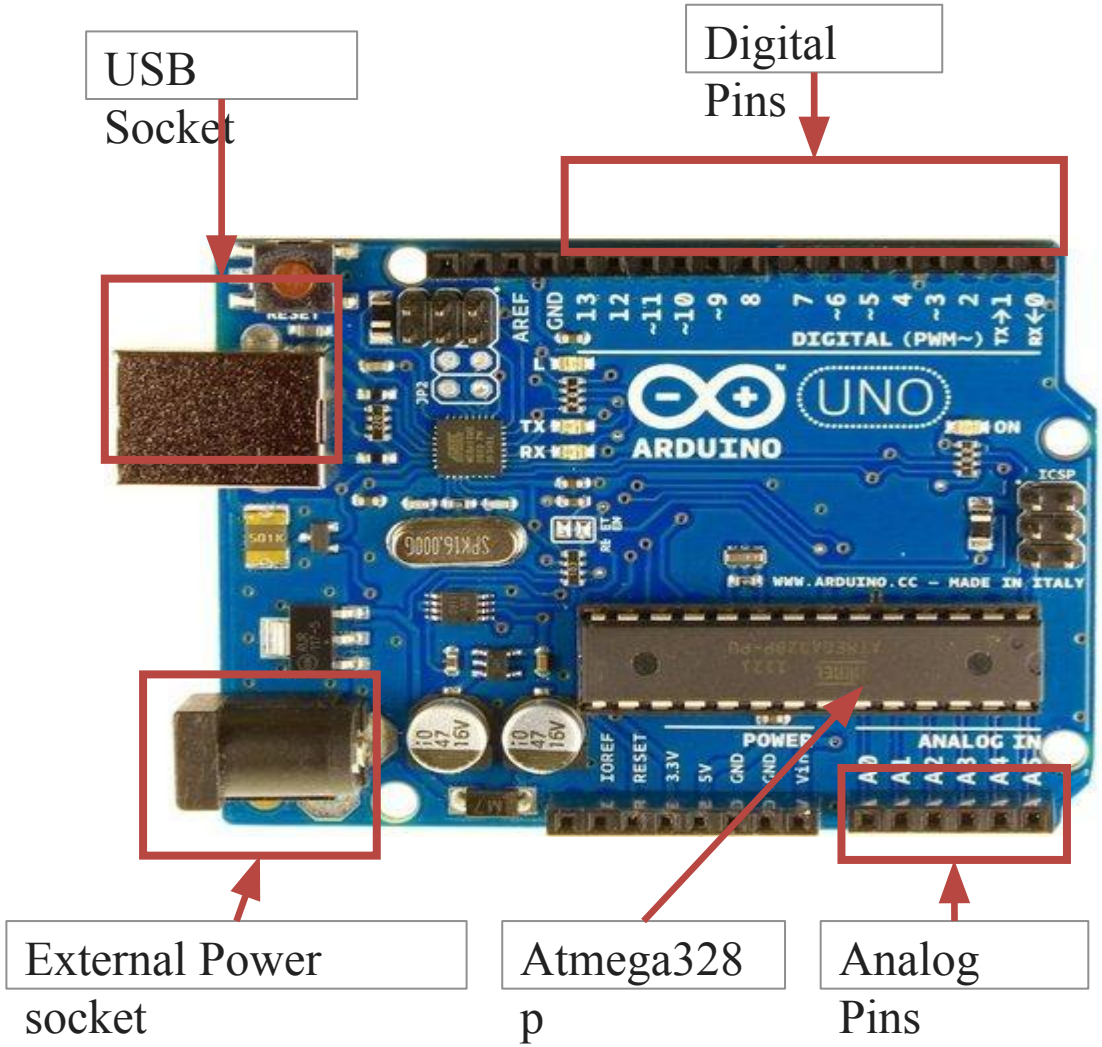
- **Microcontroller** is a digital device consisting of **CPU, peripheral I/Os, memories, interrupts,** etc. in a single chip or IC.
- Microcontroller is a **compact integrated circuit** designed for **a specific operation** in an embedded system.
- The examples of various microcontrollers are given as:

Atmel AVR	**Microchip – PIC**	**Intel 8051**
AtMega8	PIC18F452	AT89C51
AtMega328P	PIC16F877	AT89S52
AtMega16	PIC16F676	AT89c2051
AtMega2560	PIC16F72	P89V51





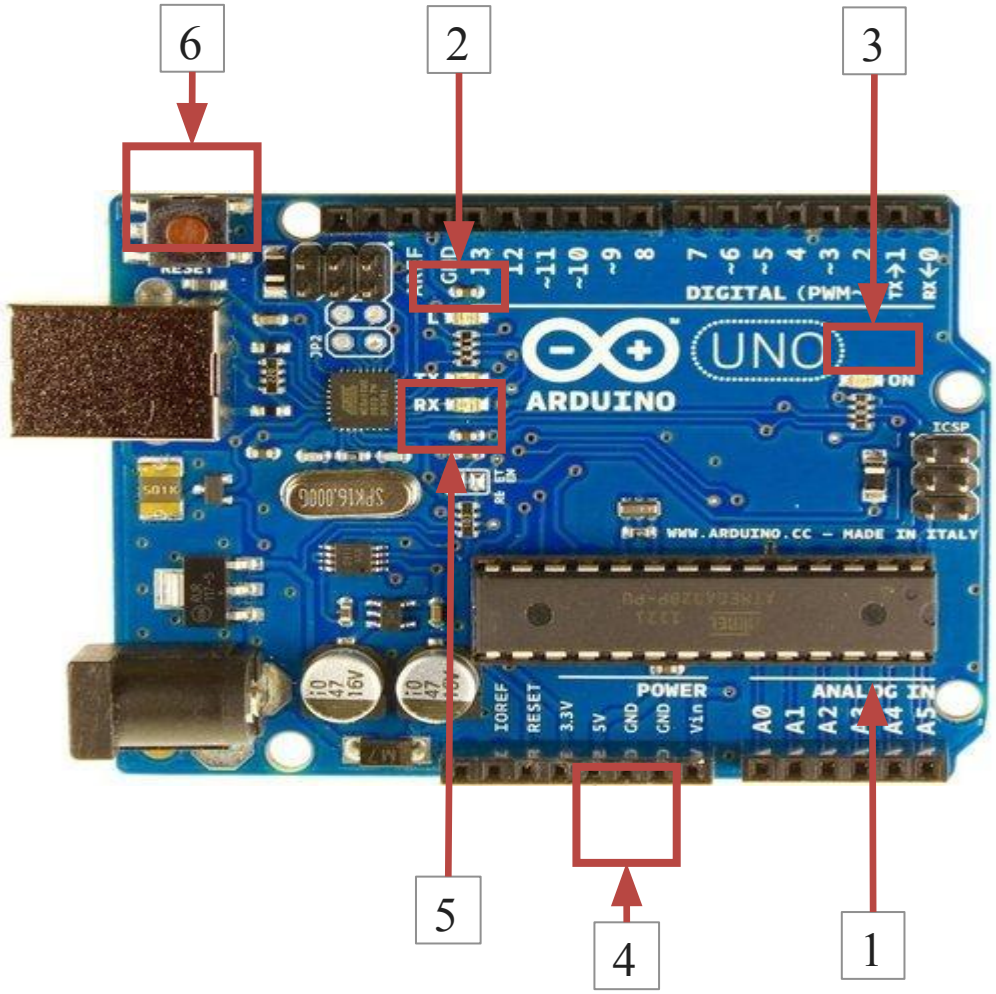
Overview of Microcontrollers



Introduction to Arduino Uno

- Atmega328p Microcontroller inside
- The operating voltage is 5V which is given by
  - USB socket or
  - External power socket
- DC Current for each input/output pin is 40 mA
- Digital input/output pins are 14 to interface digital input or output devices
- There are 6 Analog i/p pins used for interfacing analog input devices.
- What is Digital and Analog?
- Memory Components:
  - 32KB flash (program) memory
  - SRAM is 2 KB
  - EEPROM is 1 KB

Overview of Microcontrollers



Components in Arduino Uno

1. Atmega328p Microcontroller: The heart of the board.
2. On-board LED interfaced with pin 13.
3. Power LED: Indicates the status of Arduino whether it is powered or not.
4. GND and 5V pins: Used for providing +5V and ground to other components of circuits.
5. TX and RX LEDs: These LEDs indicate communication between Arduino and computer.
6. Reset button: Use to reset the ATmega328P microcontroller

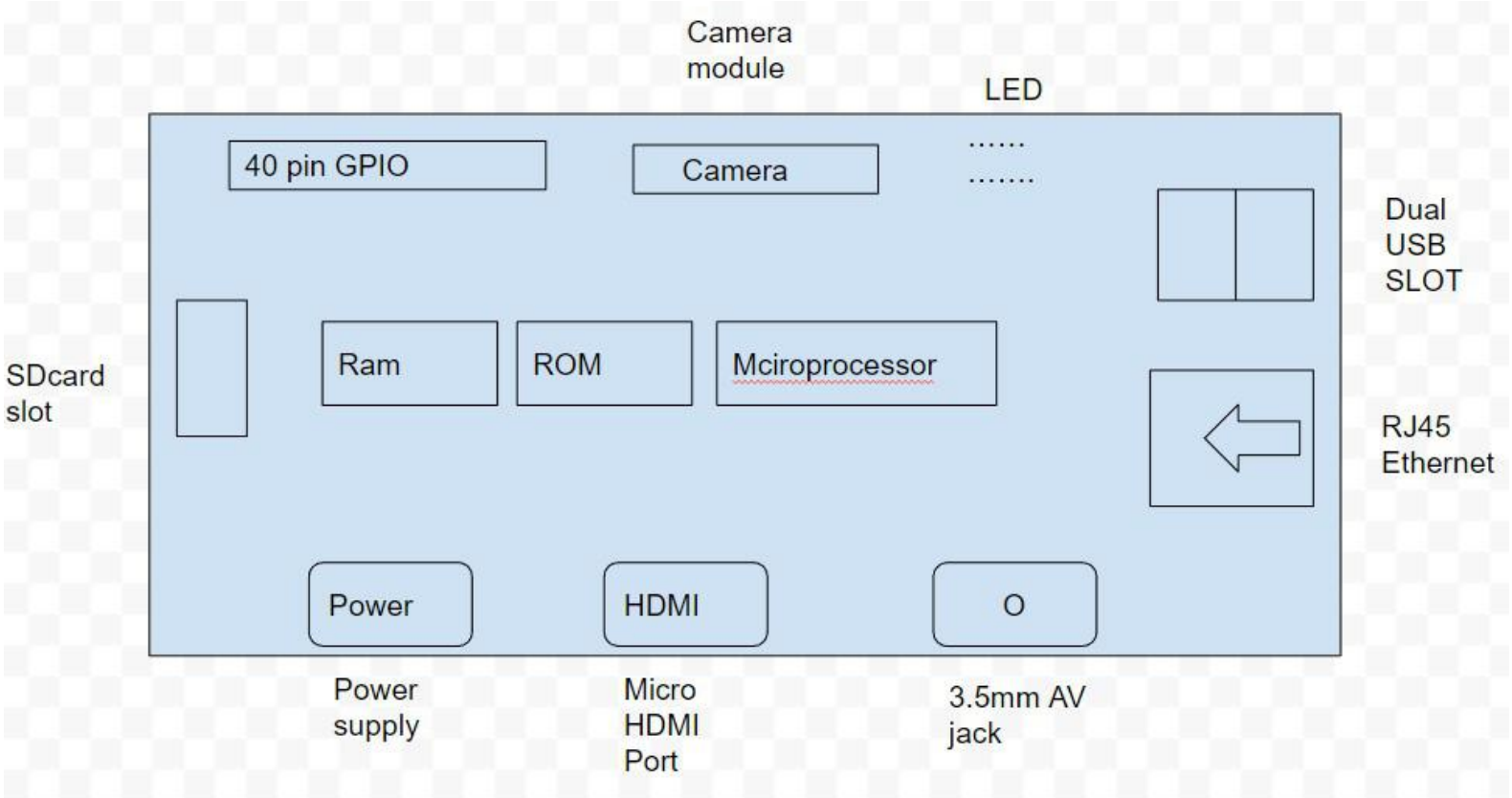
What is a Raspberry Pi

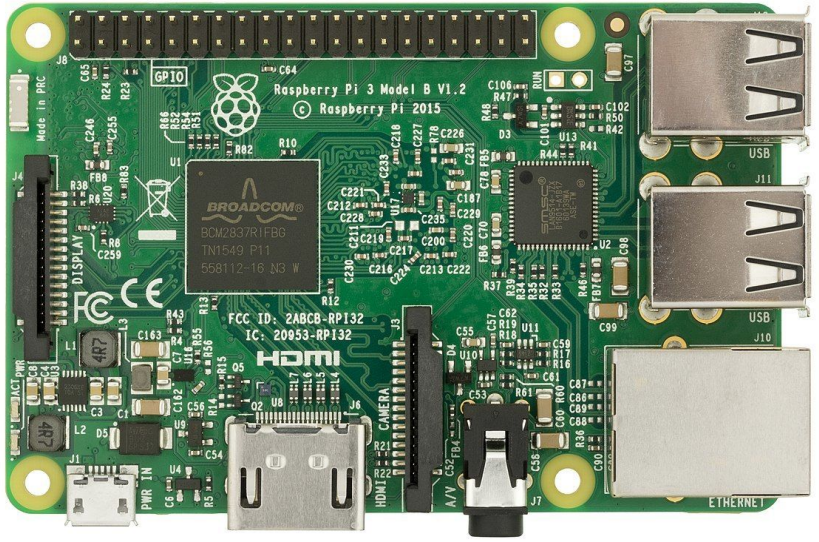
- Raspberry pi is the name of the “credit card-sized computer board” developed by the Raspberry pi foundation, based in the U.K. It gets plugged in a TV or monitor and provides a fully functional computer capability. It is aimed at imparting knowledge about computing to even younger students at the cheapest possible price. Although it is aimed at teaching computing to kids, but can be used by everyone willing to learn programming, the basics of computing, and building different projects by utilizing its versatility.
- Raspberry Pi is developed by Raspberry Pi Foundation in the United Kingdom. The Raspberry Pi is a series of powerful, small single-board computers.

- **Used:**
- It also provides a set of general purpose input/output pins allowing you to control electronic components for physical computing and explore the Internet of Things (IOT).

What is a Raspberry Pi

Raspberry pi Diagram :





Sensors and Their Interfacing

Definition of Sensor

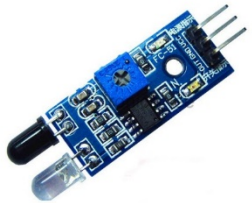
- **Sensor** is a device that detects or measures a physical property and records, indicates, or otherwise responds to it.
- A **sensor** is a device that measures **physical input** from its environment and converts it into data that can be **interpreted by either a human or a machine**.
- Normally, the sensors are input devices and there are two types of sensors.
  - **Analog Sensors**
  - **Digital Sensors**

List of Sensors

- The list of sensors which is to be covered in the chapter is as follows:

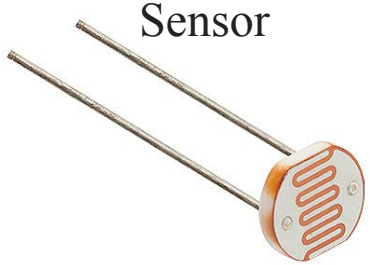
MQ-02/05 Gas Sensor

Obstacle Sensor

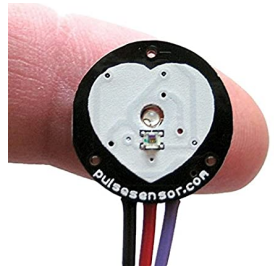


Ultrasonic Distance Sensor

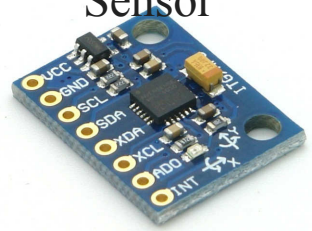
LDR Sensor



Heartbeat Sensor



Gyro Sensor



GPS Sensor

Color Sensor

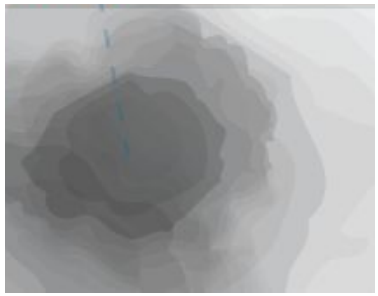
pH Sensor

MQ-02/05 Gas Sensor

- MQ -02 sensor is used to **detect smoke.**
- H2, LPG, CH4 alcohol and smoke can be detected by MQ-02.
- MQ-05 **is not sensitive to smoke**, hence less used in the application.
- Pinout of MQ-02 sensor module are
  - Vcc – Connected to 5V
  - Gnd – Connected to ground
  - D0 – Digital output pin
  - A0 – Analog output pin

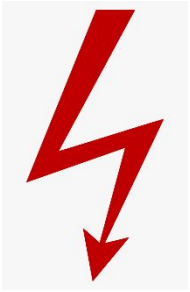
MQ-02/05 Gas Sensor – Working principle

- When any flammable gas passes through the coil of the sensor, the coil burns and **internal resistance decreases.**
- This results in an **increased voltage** across it. Hence we get variable voltage at A0 pin of MQ-05 gas sensor.
- If gas present -> voltage is high.
- If gas is not present -> voltage is low.



Smoke/Gas

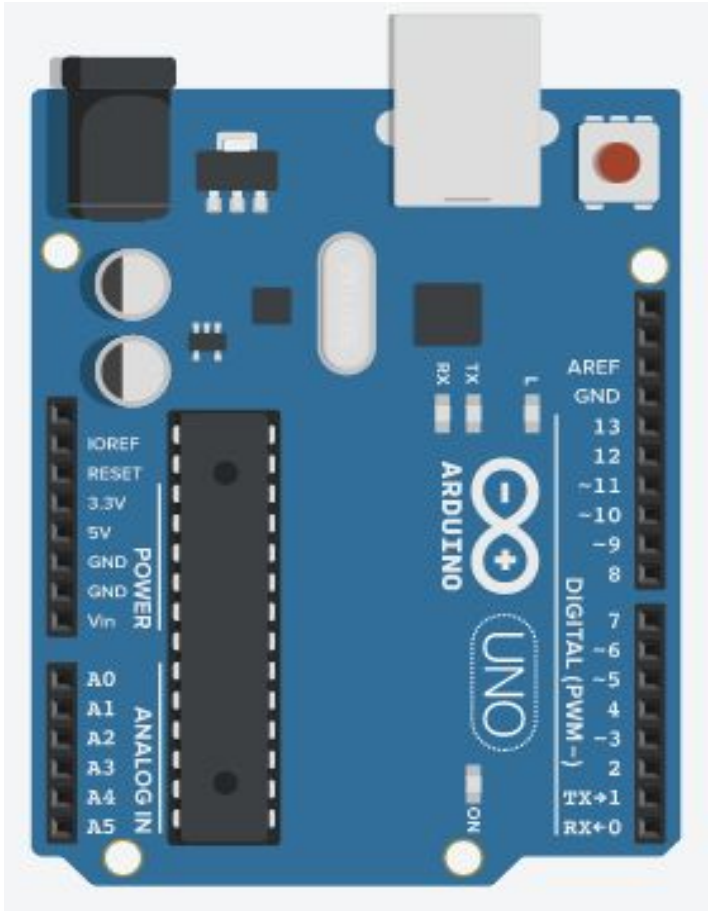
MQ-05 Gas Sensor



Variable Voltage

MQ-02/05 Gas Sensor – Interfacing with Arduino

- The interfacing of MQ-02 with Arduino Uno is as shown in the figure.
- Here, Vcc and Gnd pins of MQ-02 are connected to 5V and GND of Arduino board.
- We want to take the input from a Gas sensor in **an analog** format so pin A0 is connected to one of the analog pins of Arduino i.e. A0.



MQ-02/05 Gas Sensor – Code explanation

- The code in Arduino for MQ-02 gas sensor can be written as following

gas_sensor.ino

```
1  int gas_level;
2  void setup() {
3    pinMode(A0, INPUT);
4    Serial.begin(9600);
5  }
6
7  void loop() {
8    gas_level = analogRead(A0);
9    Serial.print("Gas Level : ");
10   Serial.println(gas_level);
11   delay(500);
12 }
```

Initialize the serial communication between Arduino and computer with baud rate 9600.

Reads the analog value from specified pin and stores it in the mentioned variable.

Prints data on the serial port in ASCII text given in double quotes.

Prints the data of specified variable on the serial port and also prints new line at the end.

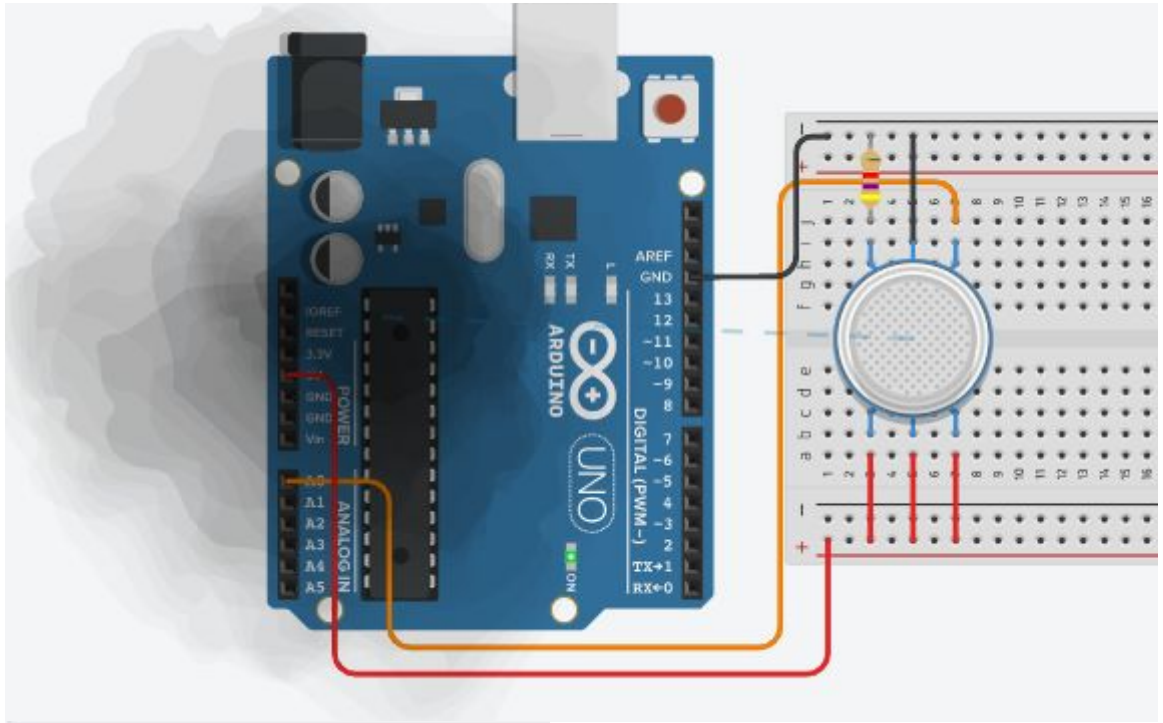
Functions in Arduino IDE

- Serial.begin( ): It is used to start serial communication between Arduino board and other device. Normally, it is connected with computer for monitoring the data. Serial communication uses pin 0 and 1 also working as Rx and Tx respectively.
  - Syntax : Serial.begin(baud_rate)
  - Parameters :
    - baud_rate: It defines the speed of data transfer rate between Arduino and other device. It is compulsory to set this baud rate same at both the side for proper transfer of data.
- Serial.print( ) : Print the data from Arduino to other device.
  - Syntax : Serial.print("text")
  - Parameters :
    - The text which is to be printed by Arduino board is written in double quote so as to convert it in its ASCII values.
- Serial.println( ) : Print the data with newline from Arduino to other device.
  - Syntax : Serial.println("text")
  - Parameters :
    - The text which is to be printed by Arduino board is written in double quote so as to convert it in its ASCII values.

Functions in Arduino IDE (Cont.)

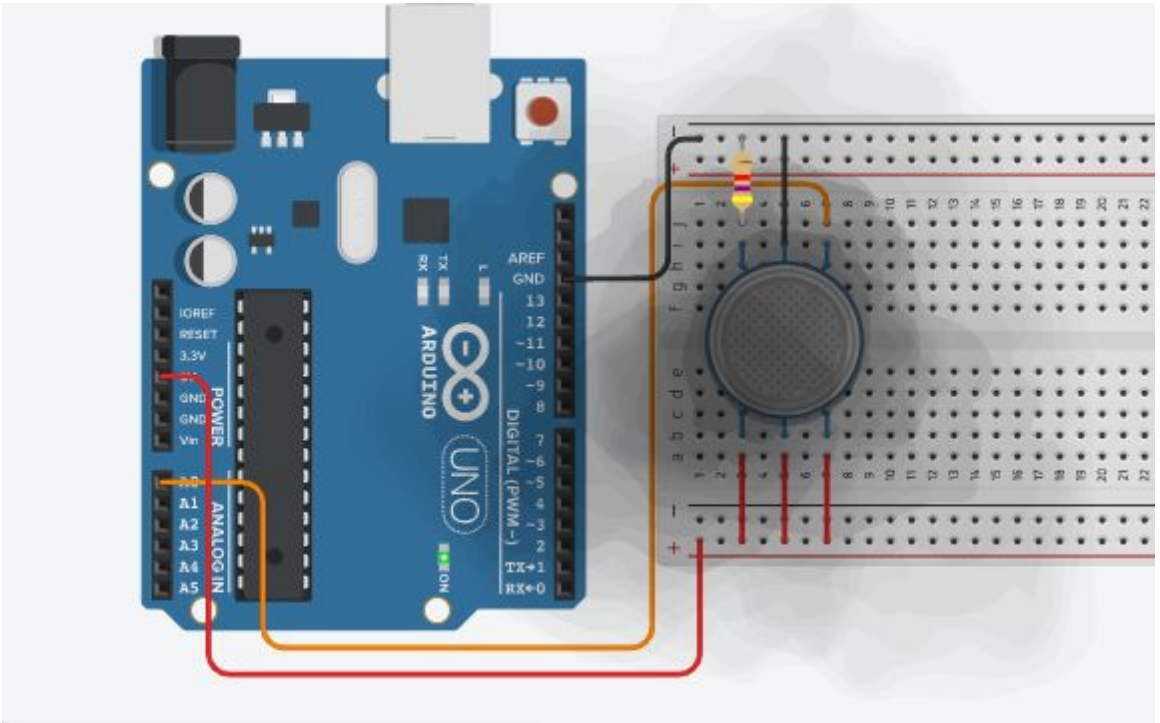
- analogRead( ): The syntax reads analog value from specified pin and converts it into 1024 levels or in 0 to 1023 range. The values is stored into specified variable in the code statement. The value is converted into 1024 levels because Arduino has 10 bit built in A-D converter unit.
  - Syntax : analogRead(pin)
  - Parameters :
    - pin: The Arduino analog pin number.

MQ-02/05 Gas Sensor – Output



Serial Monitor

```
Gas Level : 306
Gas Level : 306
Gas Level : 306
Gas Level : 306
Gas Level : 306
Gas Level : 306
```



Serial Monitor

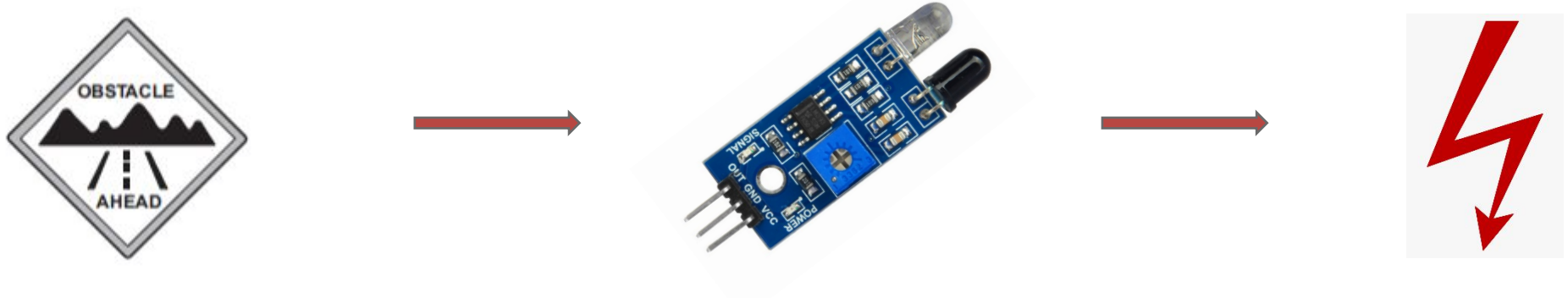
```
Gas Level : 745
Gas Level : 745
Gas Level : 745
Gas Level : 745
Gas Level : 745
Gas Level : 745
```

Obstacle Sensor

- Obstacle sensor is used for **detection of obstacle**.
- It is a digital sensor, hence gives binary output ‘1’ or ‘0’.
- The **range** for detection of obstacle can be changed by **potentiometer** given.
- Pinout of obstacle sensor module are
  - Vcc – Connected to 5V
  - Gnd – Connected to ground
  - Out/D0 – Digital output pin

Obstacle Sensor – Working principle

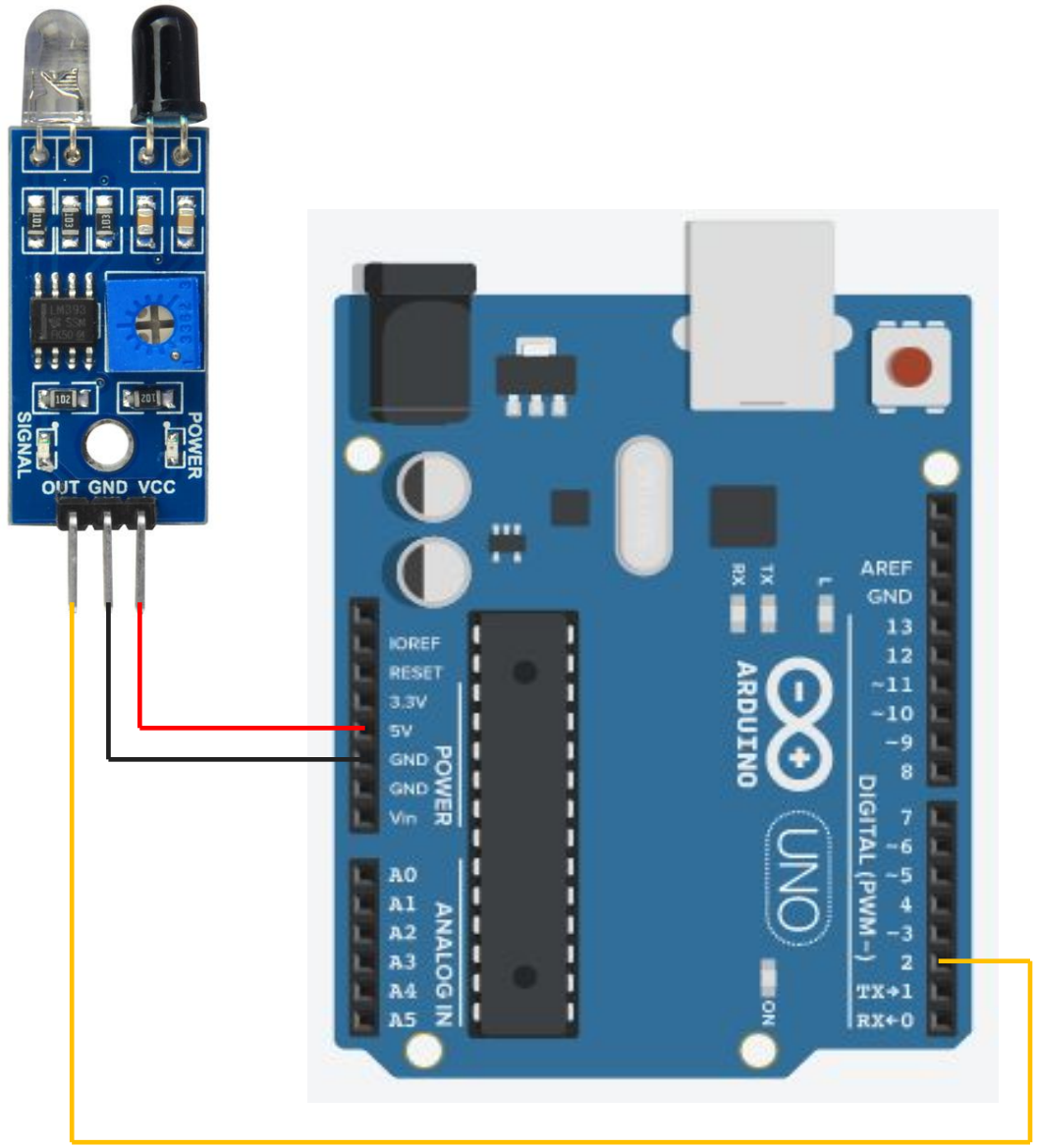
- IR emitter transmits IR signals and IR receiver receives those signals from reflection.
- If the obstacle is present then the transmitted signals are **reflected** back by obstacle and if they have amplitude greater then threshold the output signal will be **'0'**.
- If the obstacle is not present then the transmitted signals are **not reflected** and the output signal will be **'1'**.



Obstacle in the way → Obstacle IR Sensor → Digital Output

Obstacle Sensor – Interfacing with Arduino

- The interfacing of obstacle sensor with Arduino Uno is as shown in figure.
- Here, Vcc and Gnd pins of obstacle sensor is connected to 5V and Gnd of Arduino board.
- The output of obstacle sensor is in **digital** form. So it is connected to digital pin 2 of Arduino.



Obstacle Sensor – Code explanation

- The code in Arduino for Obstacle sensor can be written as following:

obstacle-detect.ino

```
1  int obstacle_pres=0;
2  void setup() {
3    pinMode(2, INPUT);
4    pinMode(13, OUTPUT);
5    Serial.begin(9600);
6  }
7
8  void loop() {
9    obstacle_pres = digitalRead(2);
10   if (obstacle_pres == LOW)
11   {
12      Serial.print("Obstacle is present");
13      digitalWrite(13,HIGH);
14   }
```

Reads the digital data from specified pin and stores it into given variable.

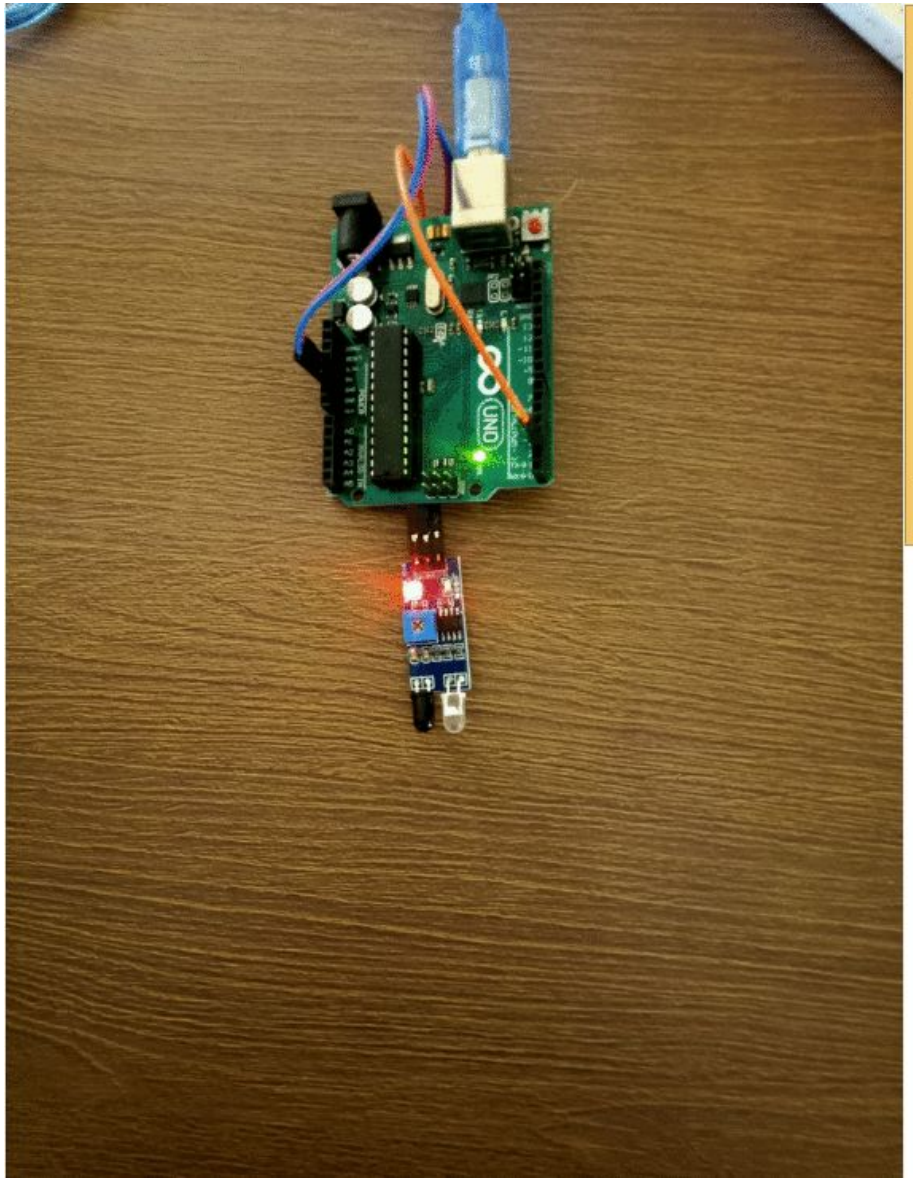
Obstacle Sensor – Code explanation (Cont.)

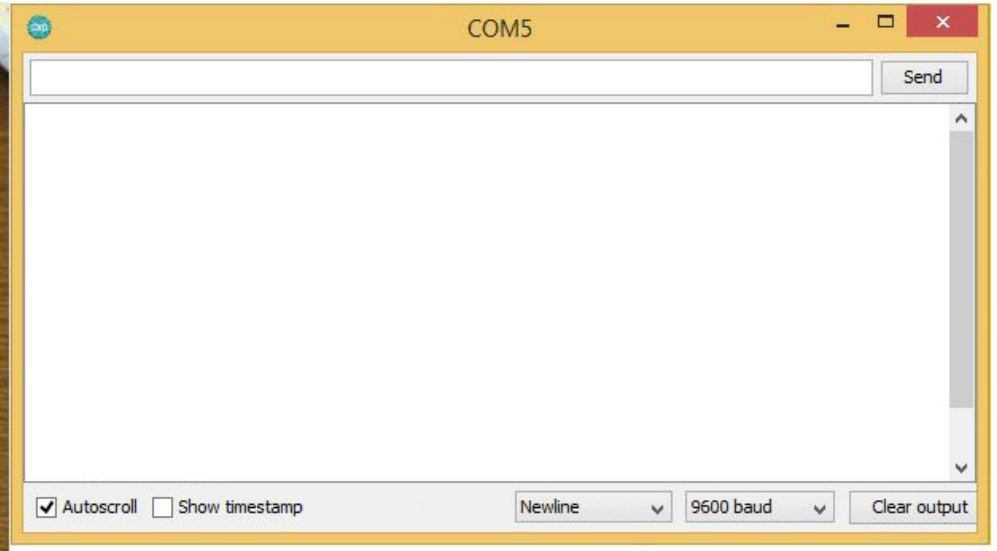
obstacle-detect.ino

```
13    else
14    {
15       digitalWrite(13,LOW);
16    }
17    delay(1000);
18 }
```

- digitalRead( ) : Reads digital data from specified pin and stores it into given variable.
  - Syntax : digitalRead(pin)
  - Parameters :
    - Pin : The Arduino digital pin number.

Obstacle Sensor – Output



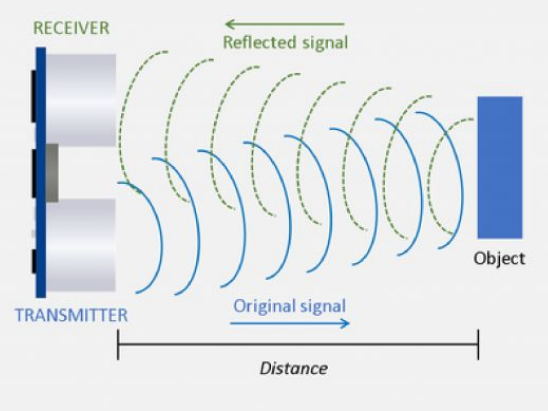


HC-SR04 Ultrasonic Sound Sensor

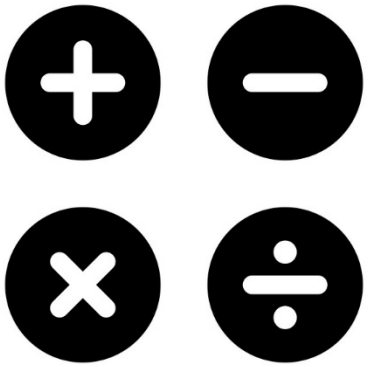
- Ultrasonic Sound Sensor is used to measure the **distance of an object.**
- The sensor can only measure the distance of object. It can not identify the type of object.
- It is also known as Ultrasonic distance sensor.
- Pinout of HC-SR04 module are
  - Vcc – Connected to 5V
  - Gnd – Connected to ground
  - Trigger – Generates the transmit pulse
  - Echo – Receives echo from obstacle

HC-SR04 Ultrasonic Sound Sensor – Working principle

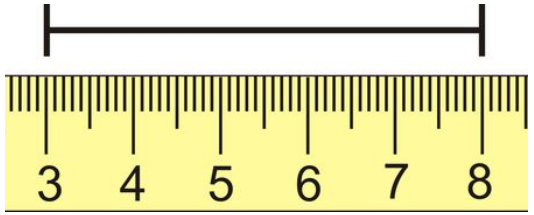
- The emitter transmits ultrasonic sound waves which are reflected by near by objects.
- The reflected pulse is received by sensor.
- Some mathematical operations are performed to obtain the distance value.
- The distance value is converted into desired unit.



Ultrasonic waves



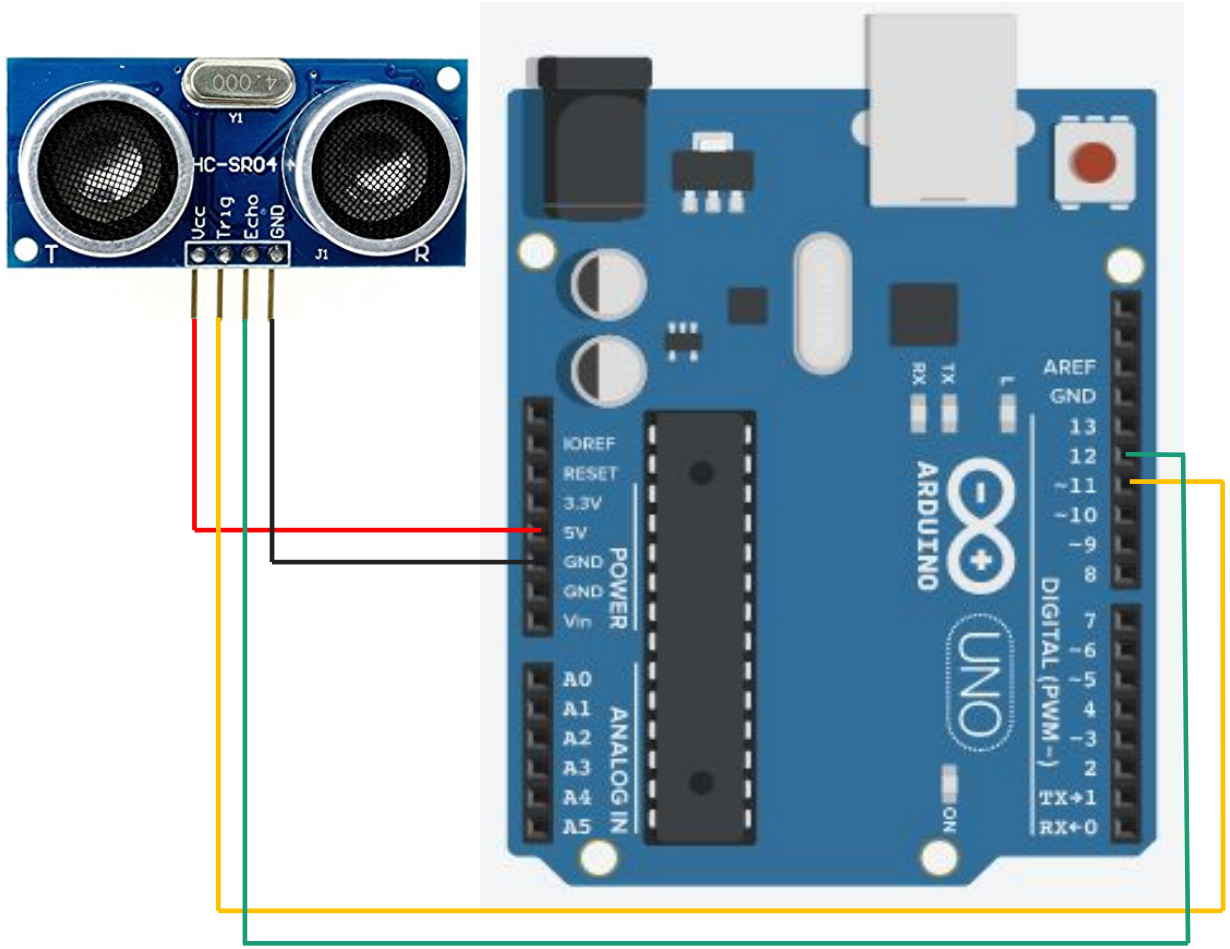
Mathematical operations



Measurement of distance

HC-SR04 Ultrasonic Sound Sensor– Interfacing with Arduino

- The interfacing of HC-SR04 sensor with Arduino Uno is as shown in figure.
- Here, Vcc and Gnd pins of HC-SR04 sensor are connected to 5V and Gnd of Arduino board.
- The trigger and echo pin is connected to (any) digital pins of Arduino board.
- In this case, trigger is connected to 11 and echo is connected to 12.



HC-SR04 Ultrasonic Sound Sensor – Code explanation

- The code in Arduino for HC-SR04 Ultrasonic Sound Sensor can be written as following

Distance-measure.ino

```
1  int trigPin = 11;     // Trigger
2  int echoPin = 12;     // Echo
3  long duration, cm, inches;
4
5  void setup() {
6    Serial.begin (9600);
7    pinMode(trigPin, OUTPUT);
8    pinMode(echoPin, INPUT);
9  }
10 void loop() {
11   digitalWrite(trigPin, LOW);
12   delayMicroseconds(5);
13   digitalWrite(trigPin, HIGH);
14   delayMicroseconds(10);
15   digitalWrite(trigPin, LOW);
```

Generates the delay of specified microseconds.

HC-SR04 Ultrasonic Sound Sensor – Code explanation (Cont.)

Distance-measure.ino

```
16    pinMode(echoPin, INPUT);
17    duration = pulseIn(echoPin, HIGH);
18
19    // Convert the time into a distance
20    cm = (duration/2) / 29.1;
21    inches = (duration/2) / 74;
22    Serial.print(inches);
23    Serial.print("in, ");
24    Serial.print(cm);
25    Serial.print("cm");
26    Serial.println();
27
28    delay(250);
29  }
```

Reads a HIGH pulse on specified pin and returns the value of time in microsecond for transition from LOW to HIGH.

Distance(m) = Speed (m/s)x Time (s)

But, Time of pulse we receive is in µs and distance we want to find is in cm.
Distance(cm) = Speed (cm/µs) x Time (µs)

Speed of sound is 343m/s which is converted into 0.0343 cm/µs.
So either we have to multiply 0.0343 or divide 29.1

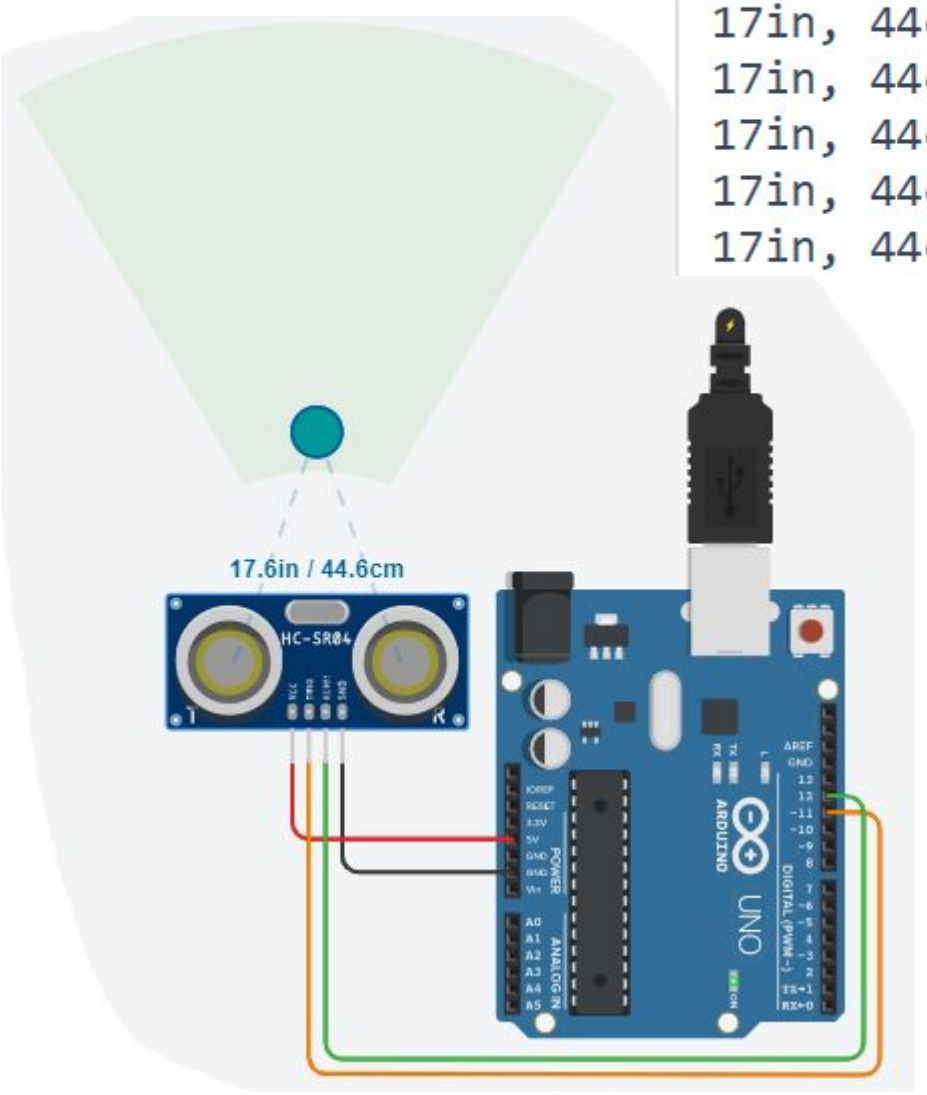
Functions in Arduino IDE

- delayMicroseconds( ): It pauses the program for amount of time in microseconds specified as the parameter.
  - Syntax : delayMicroseconds(µs)
  - Parameters :
    - µs : The number of microseconds to pause.
- pulseIn( ) : Reads a pulse HIGH or LOW from specified pin and wait for the time to go the pulse from HIGH to LOW or LOW to HIGH. It returns the time required to transit the pulse in variable.
  - Syntax : pulseIn(pin, value)
  - Parameters :
    - Pin: The number of the Arduino pin on which you want to read the pulse.
    - Value: The type of pulse that you want to read.

HC-SR04 Ultrasonic Sound Sensor – Output

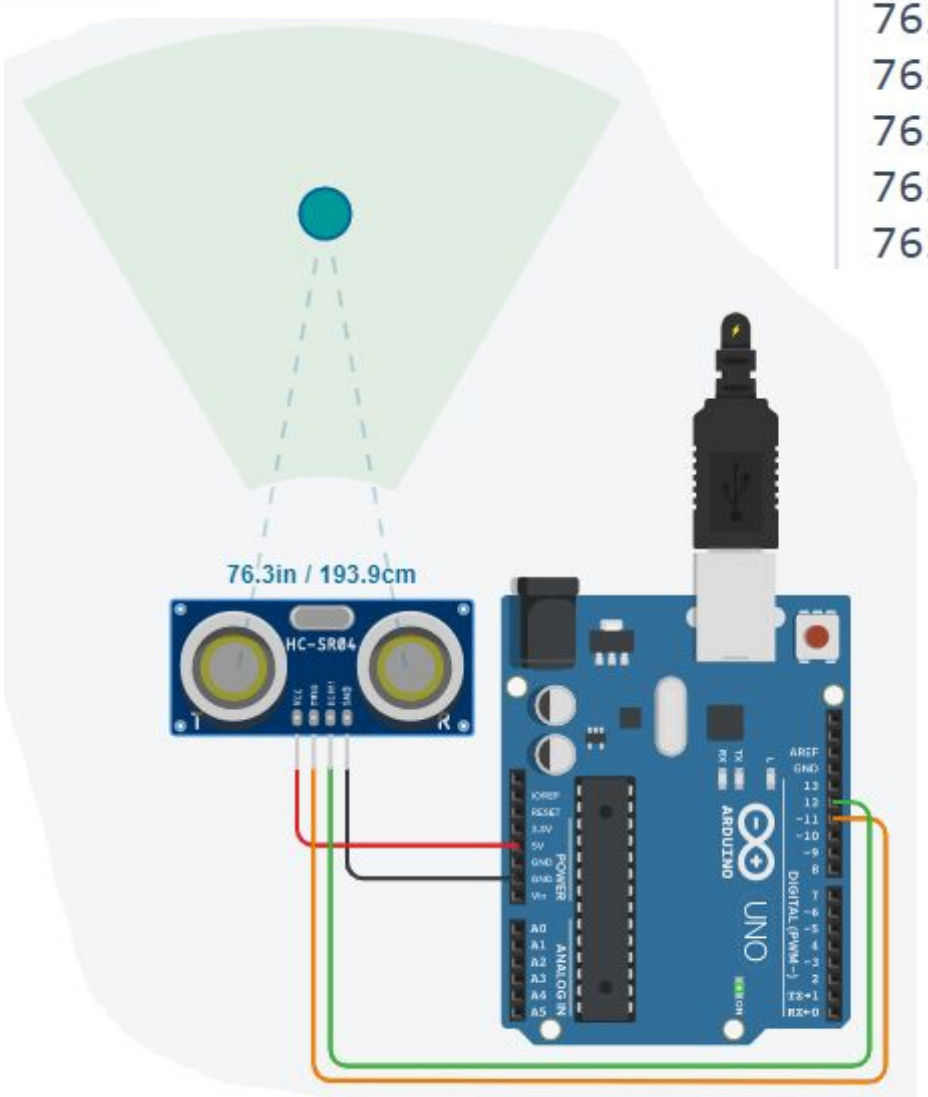
Serial Monitor

```
17in, 44cm
17in, 44cm
17in, 44cm
17in, 44cm
17in, 44cm
```



Serial Monitor

```
76in, 193cm
76in, 193cm
76in, 193cm
76in, 193cm
76in, 193cm
```

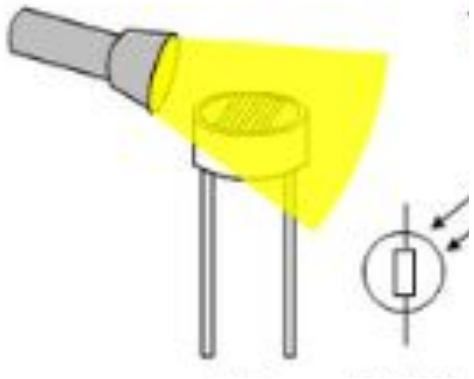


LDR Sensor

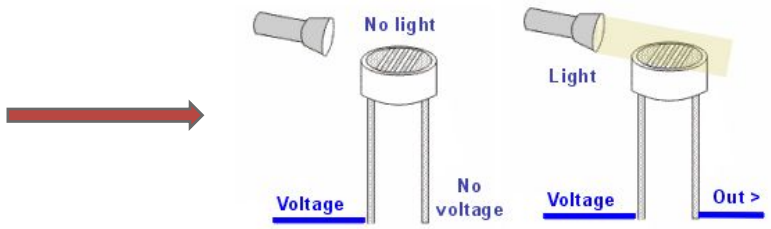
- LDR is known as Light Dependent Resistor.
- The **internal resistance** of LDR changes with respect to light intensity.
- Normally LDR works with voltage divider network.
- The LDR is used in the application where the task is performed with light intensity as input.

LDR Sensor – Working principle

- The LDR works on principle of Ohm's law.
- As the light intensity on LDR is higher, the resistance of LDR decreases.
- The decreased resistance will drop more voltage across LDR according to Ohm's Law.
- The presence of light can be identified by the value of voltage across LDR.



Light Intensity to be measured

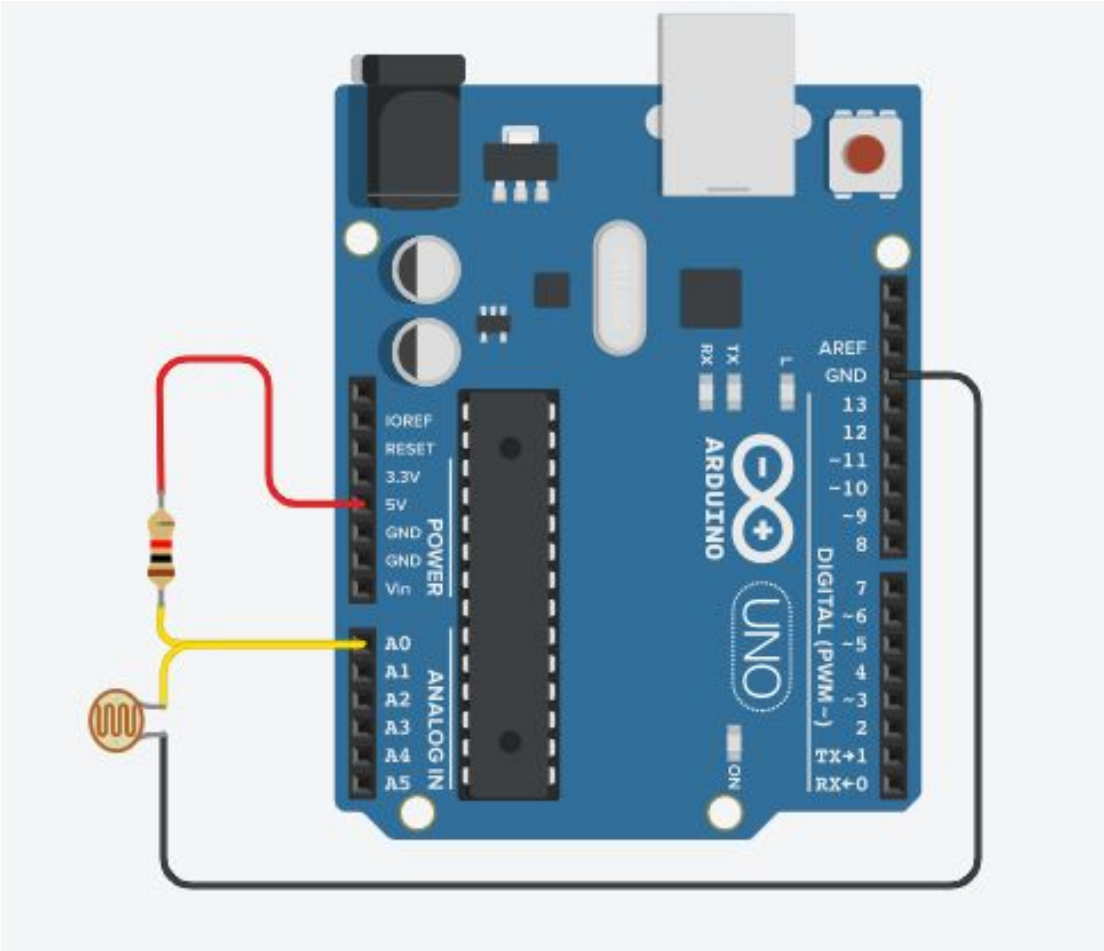


Output voltage measurement

Calculate the intensity

LDR Sensor– Interfacing with Arduino

- The interfacing of LDR sensor with Arduino Uno is as shown in figure.
- Here, one terminal of 10kΩ resistor is connected to 5V of Arduino and second terminal is connected to A0 pin.
- One terminal of LDR is connected to A0 pin of Arduino and other is connected to Gnd.
- This creates a voltage divider network between fixed resistor and LDR.
- The voltage at A0 pin remains near to 5V when LDR is in dark (high resistance).
- The voltage at A0 pin remains near to 0V when LDR is in Bright light (low resistance).



LDR Sensor – Code explanation

- The code in Arduino for LDR Sensor can be written as following

Light-intensity-measure.in
o

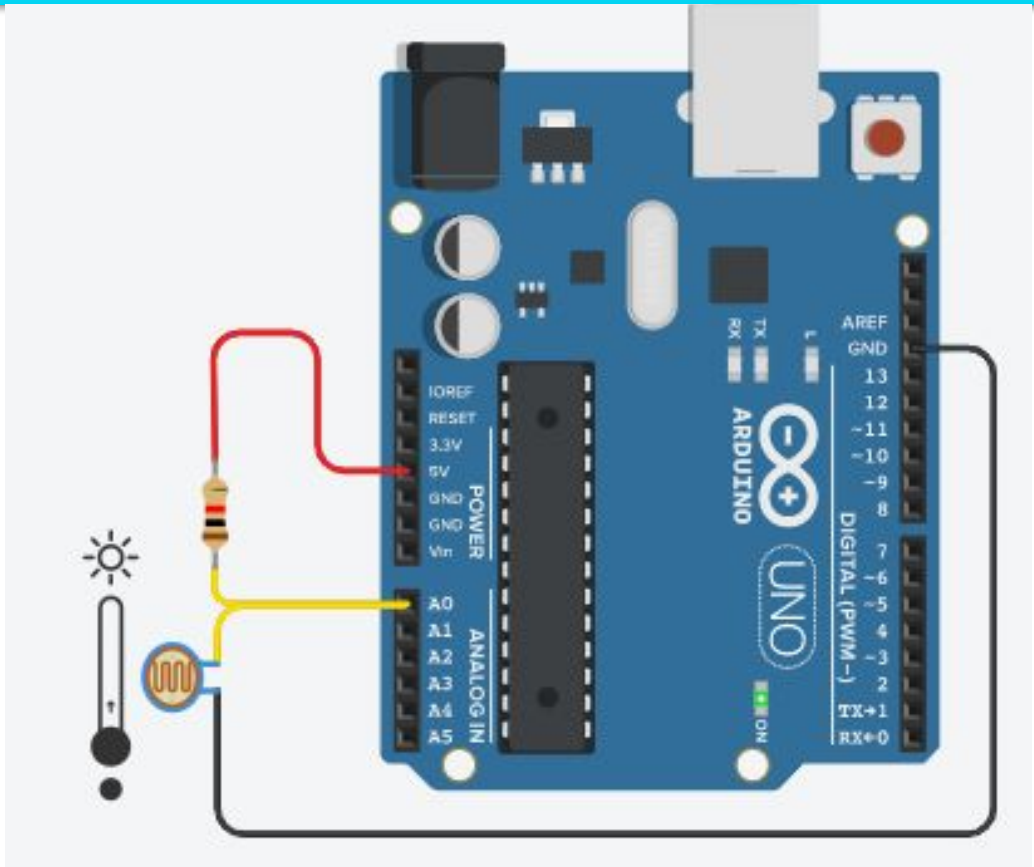
```
1  void setup()
2  {
3    pinMode(A0, INPUT);
4    Serial.begin(9600);
5  }
6
7  void loop()
8  {
9    int light_intensity = analogRead(A0);
10   if (light_intensity > 800)
11   {
12     Serial.println(“Darker");
13   }
```

LDR Sensor – Code explanation (Cont.)

Light-intensity-measure.ino

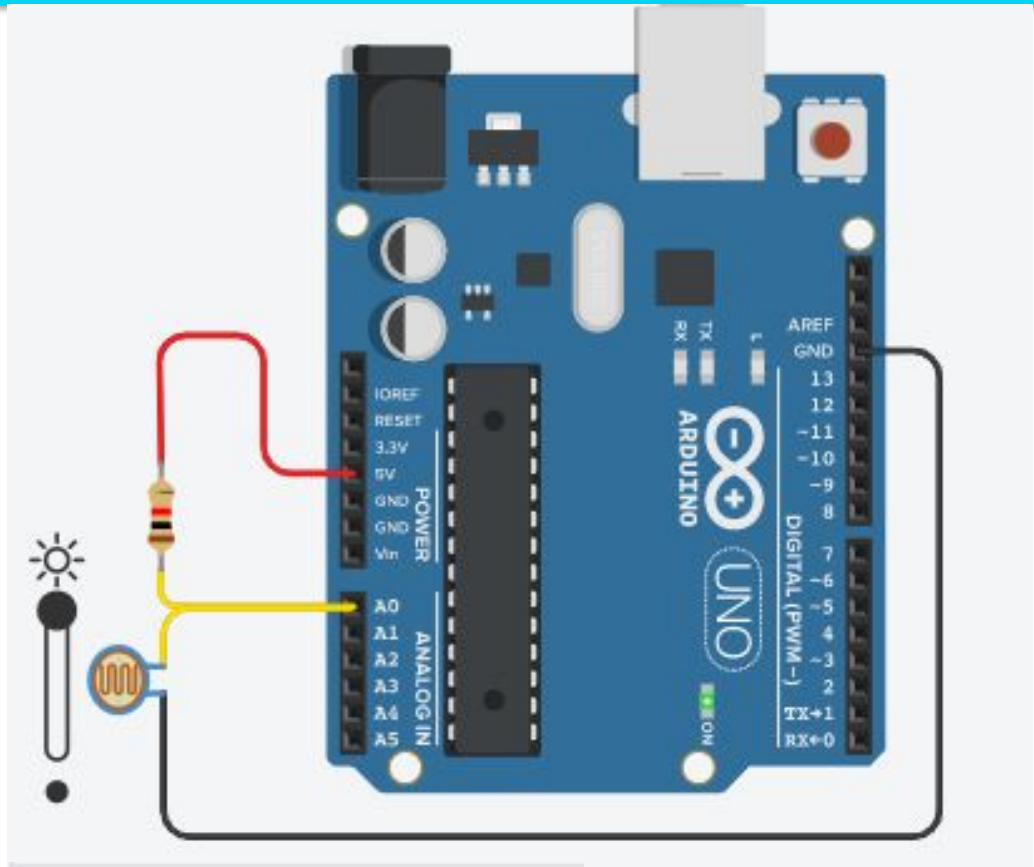
```
16    else if (light_intensity < 500)
17    {
18      Serial.println("Too Bright");
19    }
20    delay(1000);
21  }
```

LDR Sensor – Output



Serial Monitor

```
Darker
Darker
Darker
Darker
```

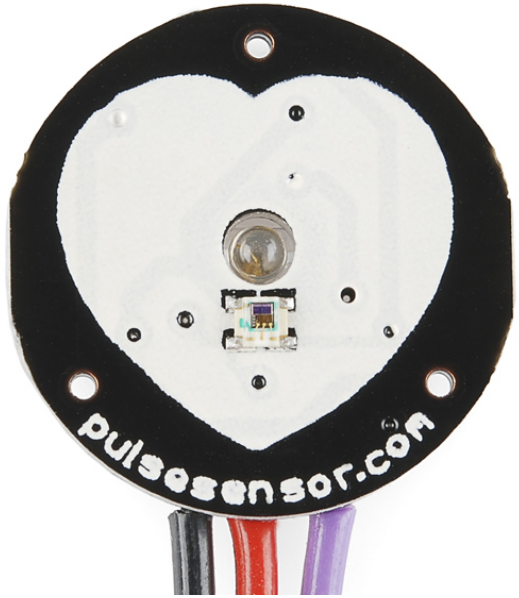


Serial Monitor

```
Too Bight
Too Bight
Too Bight
Too Bight
```

Heartbeat Sensor

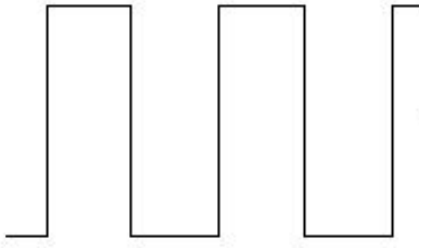
- Heartbeat sensor is having typical application in health care domain.
- This is mainly used in wearable devices to monitor the heart beat / heart rate of the person.
- The sensor is inexpensive and generates square waves for each pulse. The averaging of pulses gives analog voltage according to the heartbeat.
- Pinout of heartbeat sensor module are
  - Vcc – Connected to 5V
  - Gnd – Connected to ground
  - A0 – Analog output pin



Heartbeat Sensor – Working principle

- The sensor senses the heartbeat as per blood circulation in the body.
- It generates the output pulse signal according to the rate at which heart beats.
- The output pulses are considered as PWM waves. Hence they are converted into analog signals.
- This analog signal is converted to digital which gives the output beat rate.

Heartbeat input

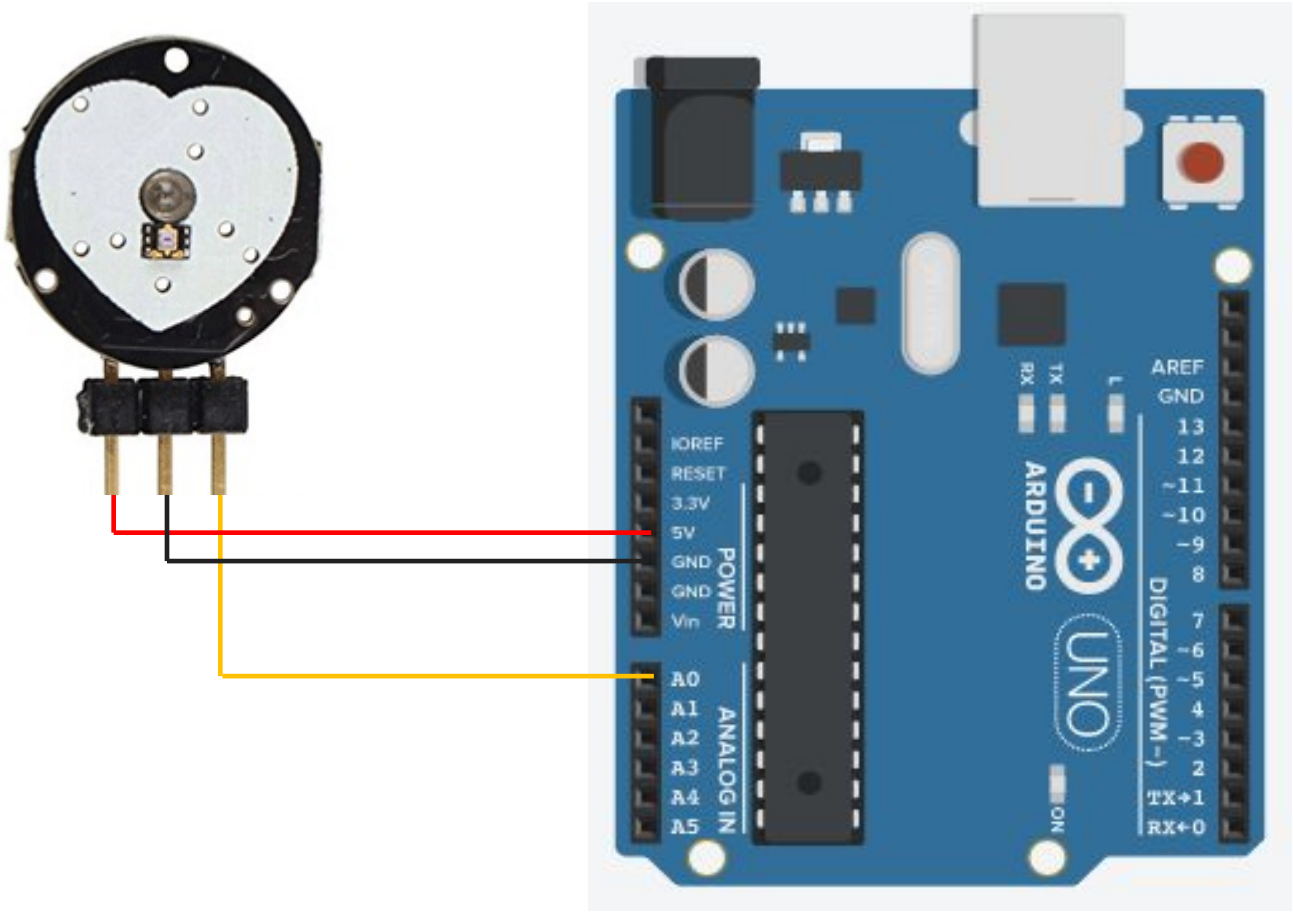


Generation of Digital Pulse as output

Calculates the BPM

Heartbeat Sensor – Interfacing with Arduino

- The interfacing of Heartbeat sensor with Arduino Uno is as shown in the figure.
- Here, Vcc and Gnd pins of heartbeat sensor are connected to 5V and GND of Arduino board.
- We want to take the input from the sensor in **an analog** format so the signal pin of the sensor is connected to one of the analog pins of Arduino i.e. A0.



Heartbeat Sensor – Code explanation

- The code in Arduino for Heartbeat sensor can be written as following:

Heartbeat_sensor.ino

```
int UpperThreshold = 518;
int LowerThreshold = 490;
int reading = 0;
float BPM = 0.0;
bool IgnoreReading = false;
bool FirstPulseDetected = false;
unsigned long FirstPulseTime = 0;
unsigned long SecondPulseTime = 0;
unsigned long PulseInterval = 0;

void setup(){
  Serial.begin(9600);
}
```

Heartbeat Sensor – Code explanation (Cont.)

Heartbeat_sensor.ino

```
14 void loop(){
15   reading = analogRead(0);
16   if(reading > UpperThreshold
17    && IgnoreReading == false){
18     if(FirstPulseDetected == false){
19       FirstPulseTime = millis();
20       FirstPulseDetected = true;
21     }
22     else{
23       SecondPulseTime = millis();
24       PulseInterval = SecondPulseTime -
25           FirstPulseTime;
26       FirstPulseTime = SecondPulseTime;
27     }
28     IgnoreReading = true;
29   }
```

millis( ) function gives the time in millisecond from where the Arduino board powered or reset.

Heartbeat Sensor – Code explanation (Cont.)

Heartbeat_sensor.ino

```
30    if(reading < LowerThreshold){
31      IgnoreReading = false;
32    }
33
34    BPM = (1.0/PulseInterval) * 60.0 * 1000;
35    Serial.print(BPM);
36    Serial.println(" BPM");
37    Serial.flush();
38  }
```

PulseInterval is in milliseconds.

$\therefore$ Rate = 1/Time

$$\therefore \text{Rate} = \frac{1}{\text{PulseInterval(ms)}}$$

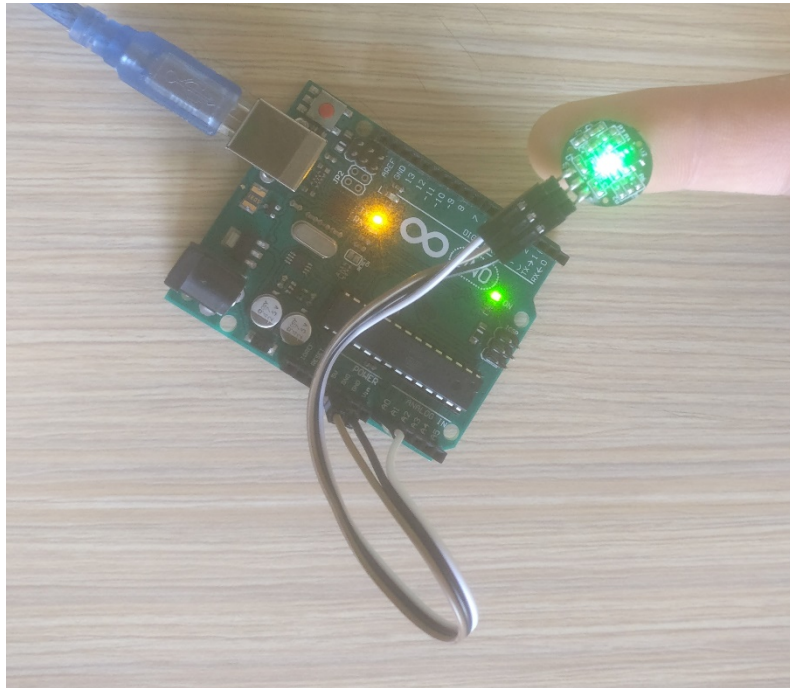
$$\therefore \text{Rate} = \frac{1}{\text{PulseInterval } (s)/1000}$$

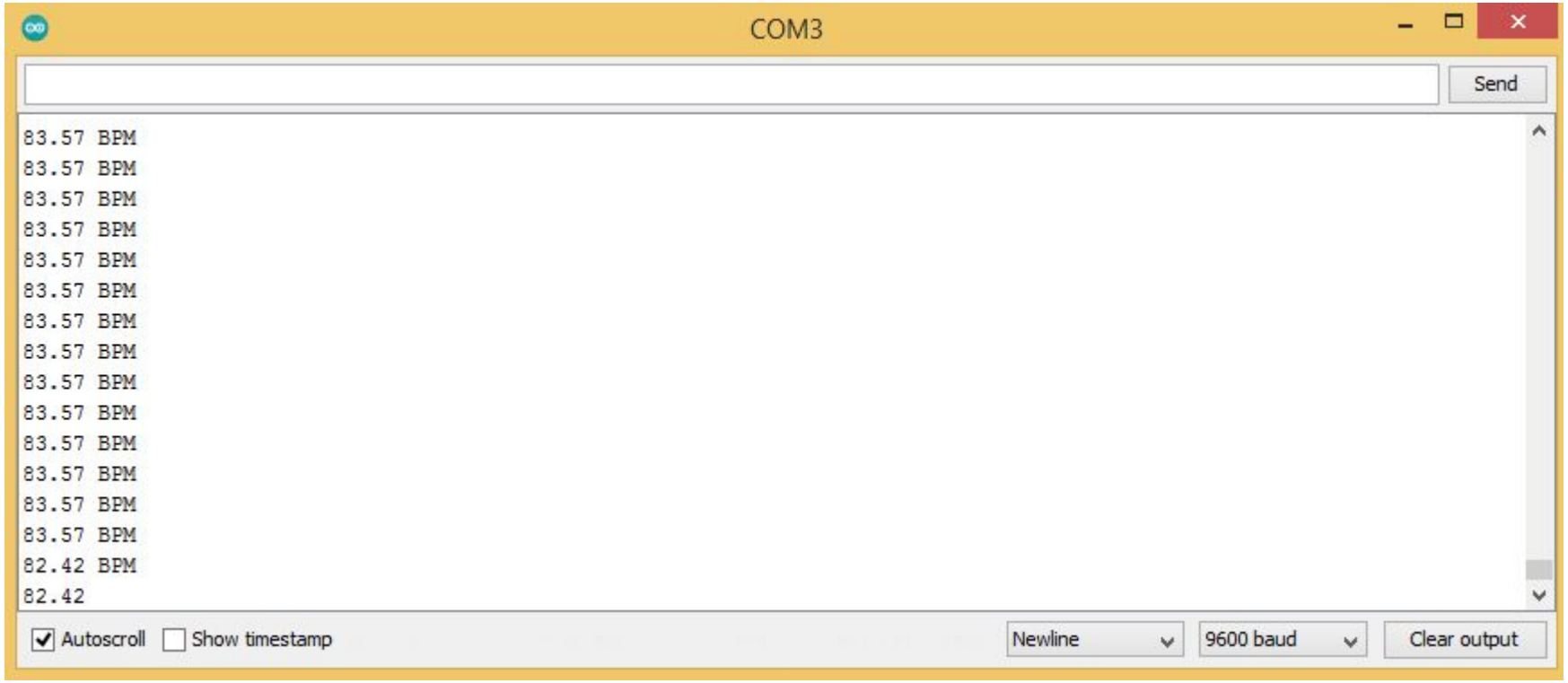
$$\therefore \text{Rate} = \frac{1}{\text{PulseInterval(s)}} * 1000$$

$$\therefore \text{Rate in min} = \frac{1}{\text{PulseInterval}} * 1000 * 60$$

- millis( ) : Returns the number of milliseconds passed since the Arduino started running the current program.
  - Syntax : millis( )

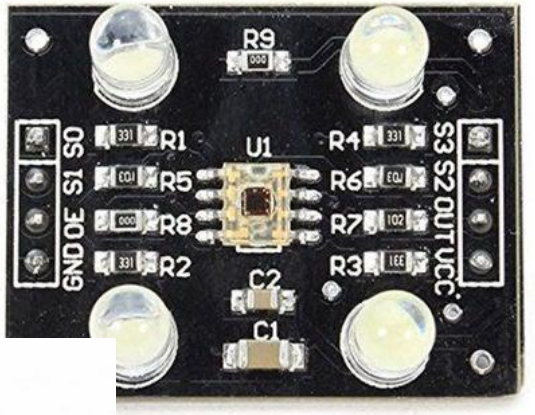
Heartbeat Sensor – Output





Colour Sensor

- Colour sensor is used to detect the RGB colour coordinates of a particular colour.
- The colour sensor module has TSC3200 IC that converts colour to frequency by enabling a particular colour diode turn by turn.
- The colour sensor is mainly used in the application where the objects are identified by their colour.
- Pinout of colour sensor module are
  - Vcc – Connected to 5V
  - Gnd – Connected to ground
  - S0, S1 – Used for output frequency scaling
  - S2, S3 – Type of photodiode selected
  - (OE)' – Enable for output
  - OUT – Output frequency (f_o)



Colour Sensor (Cont.)

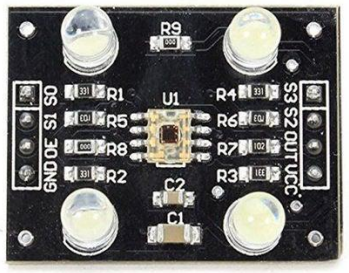
- The combination of S0 and S1 are used for output frequency scaling.
- The different microcontrollers have different counter functionality and limitations. Hence, we need to scale the frequency according to microcontroller used.
- The percentage of output scaling for different combinations of S0 and S1 are shown in the table.
- The S3 and S4 pins are used for photodiode selection. The photodiodes are associated with different colour filters.
- The table shows combinations of S3 and S4 for different photodiodes.

S0	S1	Output Frequency	Typical full-scale Frequency
L	L	Power Down	———
L	H	2%	10 – 12 KHz
H	L	20%	100 – 120 KHz
H	H	100%	500 – 600 KHz

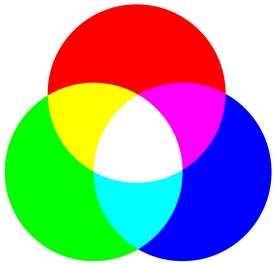
S3	S4	Photo Diode Type
L	L	Red
L	H	Blue
H	L	Clear(No Filter)
H	H	Green

Colour Sensor – Working principle

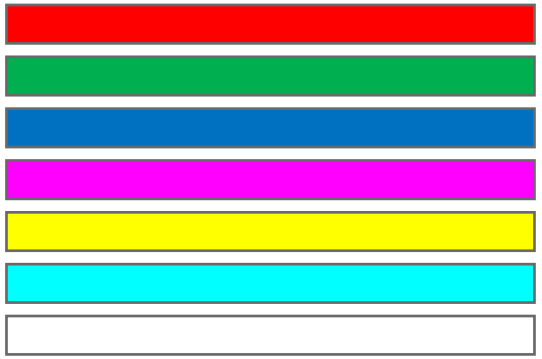
- The sensor works by imparting a bright light on an object and recording the reflected colour by the object.
- The selected photodiode for colour of red, green and blue converts the amount of light to current.
- These RGB values are further processed to identify the exact colour combination.



Input to photodiode of colour sensor



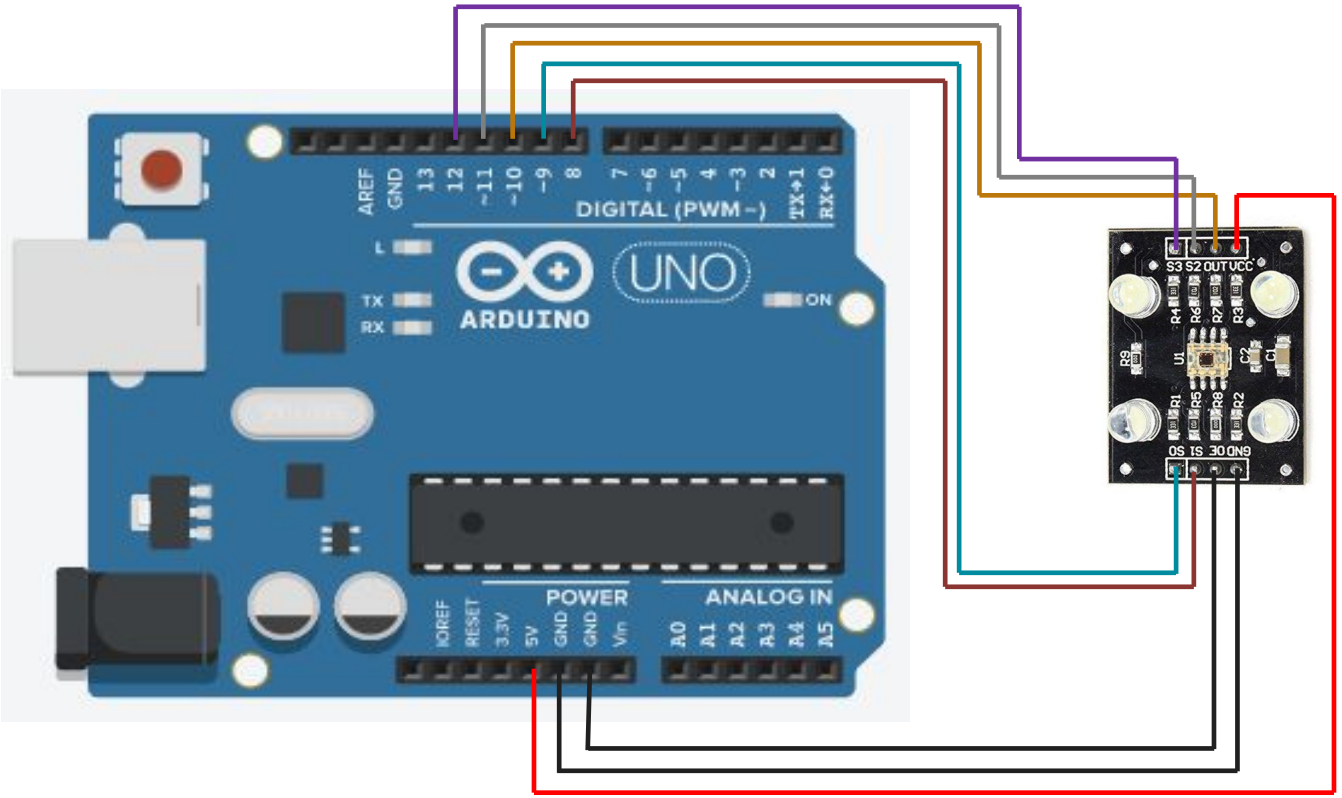
Output RGB colour values



Calculation of actual colour of object

Colour Sensor – Interfacing with Arduino

- The interfacing of Colour Sensor with Arduino Uno is as shown in figure.
- Here, Vcc and Gnd pins of Colour Sensor are connected to 5V and Gnd of Arduino board.
- As we need to enable the sensor OE' is connected to GND.
- All selection pins S0, S1 S2 and S3 are connected to digital pins of Arduino that is 9,8,11 and 12 respectively.

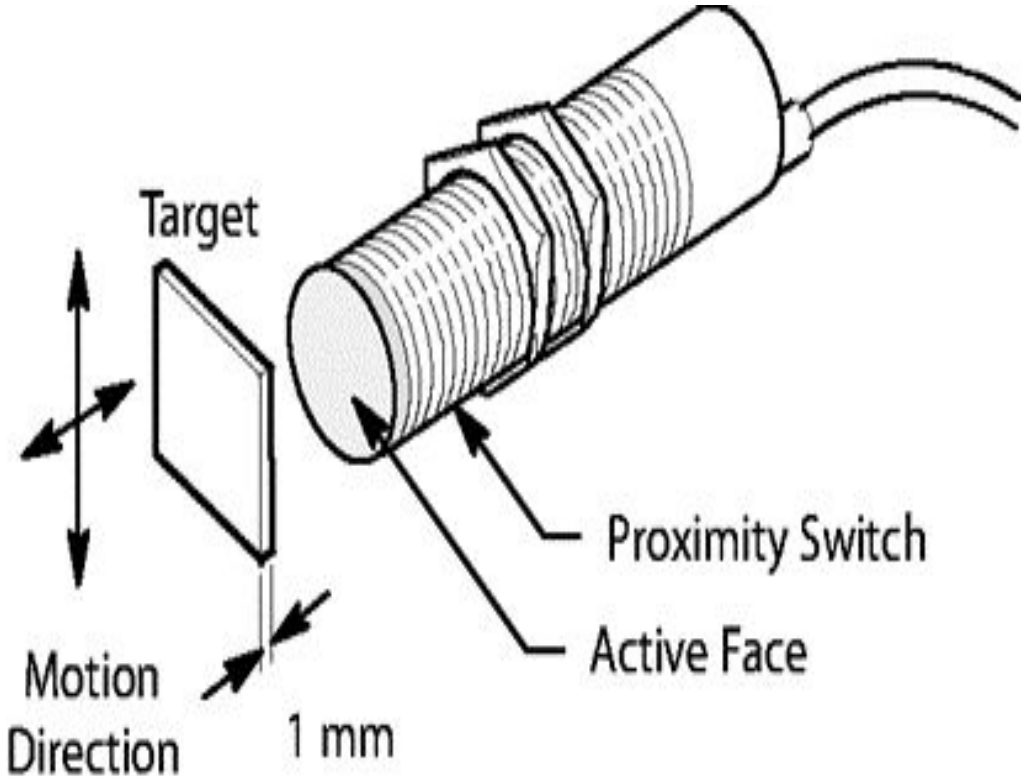


Temperature Sensor

- A Temperature Sensor is a device which is used to measure heat or temperature on the operating machine part.
- Temperature sensing is performed by gadget called Thermocouple.
- A thermocouple is a temperature-measuring device consisting of two dissimilar conductors that contact each other at one or more points.
- It produces a voltage when the temperature of one of the points differs from the reference temperature at other parts of the circuit.

Proximity Sensor

- proximity sensor is using to detect the motion and very common to use in retail shop.
  - through this device retailer will use customer's proximity to any product and same time they can sent the coupons and deals to the customer's mobile or on email.
- Now a days proximity sensor are used to check the availability or free spaces like parking space, sitting spaces in sports stadium, mall and airports.



Pressure Sensor

- A pressure sensor is a gadget equipped with a pressure-sensitive element that's used to measure the pressure of a liquid or a gas against a diaphragm made of silicon, stainless steel, etc., and converts the measured value into an electrical signal as an output.
- It's also use to measure the water flow through pipes or tank and notify the concern person when something need to be fixed.
- Now a days pressure sensor is used in aircraft and vehicles to determine the altitude and force, continuously.

Ultrasonic Sensor

- An Ultrasonic sensor is used to measure the distance between the two object by using sound waves.
- It's measure distance by sending sound wave at a specific frequency and listen that sound wave house to measure distance.
- There are two kind of an ultrasonic sensor is, "active ultrasonic sensor" and "passive ultrasonic sensors".
- An active ultrasonic sensors generates the high frequency sound wave to receive back the ultrasonic sensor for evaluating the echo.
- But, passive ultrasonic sensors are just used for detecting ultrasonic noise which is present under specific conditions.

Servo Motor

Features

- Precise position control.
- Controlled using PWM signals.
- Rotates typically from 0° to 180°.

Applications

- Robotics
- Automatic Doors
- Camera Positioning Systems

Advantages

- High Accuracy
- Easy Control
- Fast Response

Stepper Motor

Definition

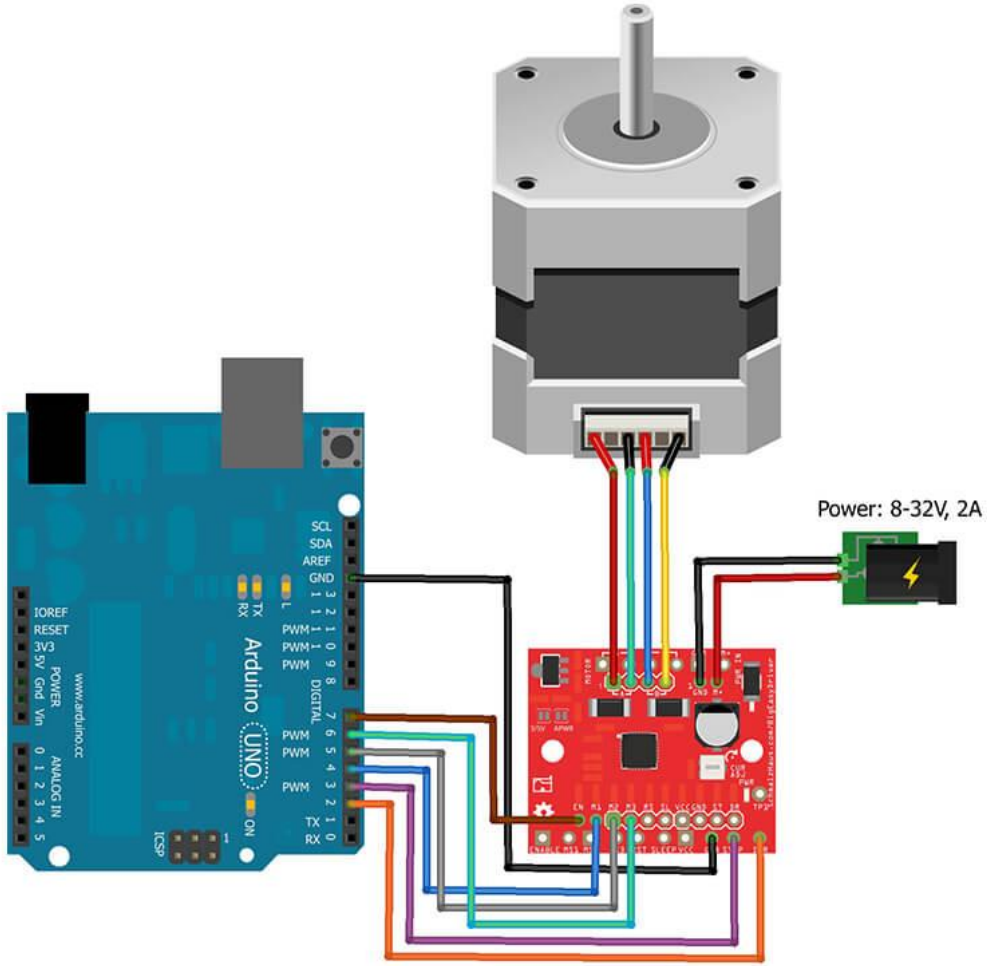
A motor that rotates in discrete steps.

Features

- High precision.
- Controlled step-by-step.
- Excellent positioning control.

Applications

- CNC Machines
- 3D Printers
- Robotics



Need of Relay While Using Actuators

A relay is an electrically operated switch that allows a low-power device such as an Arduino or Raspberry Pi to control high-power electrical devices safely.

Why is a Relay Needed?

Microcontrollers like Arduino can typically provide:

Voltage: 5V or 3.3V

Current: 20–40 mA per pin

However, actuators such as:

DC Motors

Water Pumps

Solenoids

Industrial Valves

AC Appliances

Working Principle

Arduino Pin → Relay Module → Motor/Pump/Solenoid

Built-in Constants and Data Types

Constant	Description
HIGH	Logic 1
LOW	Logic 0
INPUT	Input Pin Mode
OUTPUT	Output Pin Mode
INPUT_PULLUP	Internal Pull-up Resistor

Example

```
pinMode(13, OUTPUT);
digitalWrite(13, HIGH);
```

Data Types

Data Type	Size
boolean	1 byte
char	1 byte
int	2 bytes
long	4 bytes
float	4 bytes
double	4 bytes

Exampl
e

```
int temperature = 25;
float humidity = 65.5;
char grade = 'A';
```

Advanced Data Types

1. unsigned int: Integers without negative values, Range: 0 to 65,535 (16-bit)
unsigned int distance = 500; // Distance in millimeters

2. long: Used for large integers, Range: -2,147,483,648 to 2,147,483,647 (32-bit)
long population = 8000000000; // Earth's population

3. unsigned long: Used for large unsigned integers, Range: 0 to 4,294,967,295 (32-bit).
unsigned long uptime = 1000000; // Time in milliseconds

4. double: Similar to float (on most Arduino boards, it has the same precision as float).
double pi = 3.14159265359; // Double variable

Variables

- Variables are essential for programming as they allow your code to manipulate and use data dynamically.
- Variables are declared with a data type, a name, and optionally initialized with a value.
- Variable names must start with a letter or underscore; can include letters, numbers, and underscores but no spaces or special characters; cannot use reserved keywords.
- General syntax: `dataType variableName = value;`

Variables

Global Variables:

- Declared outside any function.
- Accessible from any function in the program.
- Typically used to store values needed throughout the program.

```
int count = 0;  // Global variable

void setup() {
  Serial.begin(9600);
}

void loop() {
  count++;  // Increment the global variable
  Serial.println(count);
  delay(1000);
}
```

Variables

Local Variables:

- Declared inside any function or bloc.
- Accessible only within that function or block.
- Used for temporary or function-specific data.

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  int localCount = 0;   // Local variable
  localCount++;         // Only accessible within this loop()
  Serial.println(localCount);
  delay(1000);
}
```

Variables

Static Variables:

- Declared inside a function with the `static` keyword.
- Retains its value between function calls.

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  static int persistentCount = 0;  // Retains its value
  persistentCount++;
  Serial.println(persistentCount);
  delay(1000);
}
```

Variables

Constants:

- Declared with the `const` keyword.
- Immutable values that do not change during program execution.

```
const int ledPin = 13;  // Constant variable

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  digitalWrite(ledPin, HIGH);
  delay(1000);
  digitalWrite(ledPin, LOW);
  delay(1000);
}
```

Variables

Variable Initialization and Assignment:

Declare and Initialize Separately:

```
int value;   // Declaration

value = 5;   // Assignment
```

Declare and Initialize Together:

```
int value = 5;  // Declaration and initialization
```

Modify Variable Values:

```
value = value + 10;  // Modify variable
```

Operators

Arithmetic Operators

1. Arithmetic Operators: Perform basic mathematical operations.

Operator	Description	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus (remainder)	x % y

1. Arithmetic Operators (contd.)

```
void setup() {
  Serial.begin(9600);
  int a = 10, b = 3;
  Serial.println(a + b);   // Output: 13
  Serial.println(a - b);   // Output: 7
  Serial.println(a * b);   // Output: 30
  Serial.println(a / b);   // Output: 3
(integer division)
  Serial.println(a % b);   // Output: 1
}
void loop() {}
```

Operators

- **Assignment Operators:** Assign or update the value of variables.

Operator	Description	Example
=	Assign value	x = 5
+=	Add and assign	x += 5
-=	Subtract and assign	x -= 5
*=	Multiply and assign	x *= 5
/=	Divide and assign	x /= 5
%=	Modulus and assign	x %= 5

Operators

- **Assignment Operators (contd.)**

```
void setup() {
  Serial.begin(9600);
  int x = 10;
  x += 5;  // x = x + 5
  Serial.println(x);  // Output: 15
}

void loop() {}
```

Operators

2. Relational Operators: Compare two values. They return `true` (1) or `false` (0).

Operator	Description	Example
==	Equal to	x == y
!=	Not equal to	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater or equal	x >= y
<=	Less or equal	x <= y

Operators

```
void setup() {
  Serial.begin(9600);
  int a = 10, b = 20;

  Serial.println(a > b);  // Output: 0 (false)

 }Serial.println(a < b);  // Output: 1 (true)

void loop() {}
```

Operators

3. Logical Operators: perform logical operations.

Operator	Description
&&	Logical AND (both true)
||	Logical OR (either one is true)
!	Logical NOT (invert truth value)

```
void setup() {
  Serial.begin(9600);
  int a = 10, b = 20;
  Serial.println((a < b) && (a > 5));  // Output: 1 (true)
  Serial.println((a > b) || (a > 5));  // Output: 1 (true)
  Serial.println(!(a < b));            // Output: 0 (false)
}

void loop() {}
```

Operators

3. Increment and Decrement Operators: Used to increase or decrease a variable's value by 1.

Operator	Description
++	Increment by 1 (++x or x++)
--	Decrement by 1 (--x or x--)

```
void setup() {
  Serial.begin(9600);
  int x = 10;
  Serial.println(x++);   // Output: 10 (post-increment)
  Serial.println(++x);   // Output: 12 (pre-increment)
}
void loop() {}
```

Control Statements

Control Statements:

Used to manage the flow of execution in a program. They help in decision-making, looping, and controlling the overall structure of the code.

1. Decision-Making Statements

- if
- if-else
- else if
- switch-case

2. Looping Statements

- for
- while
- do-while

3. Jump Statements

- break
- continue
- return

Control Statements

`if` Statement: Executes a block of code if a condition is `true`

```
void setup() {
  pinMode(13, OUTPUT);
  pinMode(7, INPUT);
}

void loop() {
  if (digitalRead(7) == HIGH) {  // If button is pressed
    digitalWrite(13, HIGH);      // Turn on LED
  }
}
```

Control Statements

`If-else` Statement: Executes one block if the condition is `true`, and another block if the condition is `false`

```
void setup() {
  pinMode(13, OUTPUT);
  pinMode(7, INPUT);
}

void loop() {
  if (digitalRead(7) == HIGH) {  // If button is pressed
    digitalWrite(13, HIGH);      // Turn on LED
  } else {
    digitalWrite(13, LOW);       // Turn off LED
  }
}
```

Control Statements

`else if` Statement: Checks multiple conditions

```
void setup() {
  pinMode(9, OUTPUT);  // LED connected to PWM pin
}

void loop() {
  int sensorValue = analogRead(A0);  // Read sensor value
  if (sensorValue < 300) {
    analogWrite(9, 50);  // Low brightness
  } else if (sensorValue < 700) {
    analogWrite(9, 150);  // Medium brightness
  } else {
    analogWrite(9, 255);  // High brightness
  }
}
```

Control Statements

`Switch-case` Statement: Selects one of many blocks of code to execute based on a variable's value

```
//LED colour selection
void setup() {
  pinMode(3, OUTPUT);  // Red
  pinMode(5, OUTPUT);  // Green
  pinMode(6, OUTPUT);  // Blue
}

void loop() {
  int mode = analogRead(A0) / 341;  // Map sensor value to 0-2
  switch (mode) {
    case 0:
      digitalWrite(3, HIGH);  // Red ON
      digitalWrite(5, LOW);
      digitalWrite(6, LOW);
      break;
    case 1:
      digitalWrite(3, LOW);
      digitalWrite(5, HIGH);  // Green ON
      digitalWrite(6, LOW);
      break;
    case 2:
      digitalWrite(3, LOW);
      digitalWrite(5, LOW);
      digitalWrite(6, HIGH);  // Blue ON
      break;
  }
}
```

Control Statements

`for` Loop Statement: Repeats a block of code a specific number of times.

```
//LED Blink Sequence
void setup() {
  for (int i = 2; i <= 7; i++) {  // Set
pins 2-7 as outputs
    pinMode(i, OUTPUT);
  }
}

void loop() {
  for (int i = 2; i <= 7; i++) {  //
Iterate over pins
    digitalWrite(i, HIGH);
    delay(500);
    digitalWrite(i, LOW);
    delay(500);
  }
}
```

Control Statements

`while` Loop Statement: Repeats a block of code as long as a condition is `true`.

```
//Wait for Button Press
void setup() {
  pinMode(7, INPUT);
}

void loop() {
  while (digitalRead(7) == LOW) {
    // Do nothing, wait for button press
  }
  Serial.println("Button Pressed!");
}
```

Control Statements

`do-while` Loop Statement: Repeats a block of code as long as a condition is `true`.

```
//LED Blinking Until Button Pressed
void setup() {
  pinMode(13, OUTPUT);
  pinMode(7, INPUT);
}

void loop() {
  do {
    digitalWrite(13, HIGH);
    delay(500);
    digitalWrite(13, LOW);
    delay(500);
  } while (digitalRead(7) == LOW);  // Stop when button is pressed
}
```

Control Statements

`break` Statement: Exits a loop or `switch-case`.

```
void setup() {
  for (int i = 0; i < 10; i++) {
    if (i == 5) {
      break;  // Exit loop when i equals 5
    }
    Serial.println(i);
  }
}

void loop() {}
```